Université de Haute-Alsace **Informatique et Réseaux - 1ère année**
ENSISA **1er semestre**

Algorithme et structure de données
Examen 1

Nom : ..... / Prénom : .....

Consignes

Aucun document autorisé, seulement une feuille format A4 recto-verso.
La calculatrice n'est pas autorisée.
Les réponses sont à mettre sur la feuille de sujet.
Durée : **1 heure**

---

1 Questions à choix multiples (20 points)

— Une seule bonne réponse par question
— Pas de point négatif pour d'éventuelles mauvaises réponses
— 1 point par question, 20 questions au totale

1.1 Compréhension des algorithmes et analyse de complexité

1. Un algorithme a résout un problème de taille n en divisant le problème en deux sous-problèmes de taille $n/2$, et combine leurs résultats en temps linéaire. Quelle est sa complexité asymptotique ?
   - ◯ $\mathcal{O}(n)$
   - ◯ $\mathcal{O}(n \log n)$
   - ◯ $\mathcal{O}(n^2)$
   - ◯ $\mathcal{O}(\log n)$
2. Considérez l'algorithme suivant :

```
for i ← 1 to n do
    j ← i
    while j < n do
        j ← j × 2
    end while
end for
```

   Quelle est sa complexité temporelle asymptotique ?
   - ◯ $\mathcal{O}(n)$
   - ◯ $\mathcal{O}(n^2)$
   - ◯ $\mathcal{O}(\log n)$
   - ◯ $\mathcal{O}(n \log n)$

3. Considérez l'algorithme suivant :

```
function M(n)
    s ← 0
    for i ← 1 to n do
        for j ← 1 to i² do
            s ← s + 1
        end for
    end for
end function
```

On sait que la complexité temporelle de cet algorithme est $\mathcal{O}(n^3)$. Quelle modification unique permettrait d'obtenir une complexité $\mathcal{O}(n^2)$?

- ◯ Remplacer la condition $j \leftarrow 1$ `to` i^2 par $j \leftarrow 1$ `to` i
- ◯ Déplacer l'instruction $s \leftarrow 0$ à l'intérieur de la boucle interne
- ◯ Remplacer la boucle externe par `for` $i \leftarrow 1$ `to` $\sqrt{n}$
- ◯ Changer $s \leftarrow s + 1$ en $s \leftarrow s + i$

4. On sait qu'un algorithme a une complexité temporelle de $\mathcal{O}(n\sqrt{n})$. Lequel des fragments de pseudocode suivants correspond le mieux à cette complexité ?

◯
```
for i ← 1 to n do
    for j ← 1 to n do
        faire une opération en O(1)
    end for
end for
```

◯
```
for i ← 1 to n do
    j ← 1
    while j × j < n do
        faire une opération en O(1)
        j ← j + 1
    end while
end for
```

◯
```
if n ≤ 1 then
    return
end if
diviser le problème en √n sous-problèmes de taille n/√n
résoudre récursivement chaque sous-problème
fusionner les résultats en temps linéaire
```

◯
```
for i ← 1 to √n do
    for j ← 1 to i do
        faire une opération en O(n)
    end for
end for
```

5. Considérez l'algorithme suivant :

```
sum ← 0
for i ← 1 to n do
    for j ← 1 to i do
        if j == i then
            sum ← sum + i
        end if
    end for
end for
```

Quelle expression décrit le mieux la valeur finale de `sum` après l'exécution ?

◯ n
◯ $\frac{n(n+1)}{2}$
◯ n^2
◯ $2n$

6. On veut calculer x^n efficacement en $\mathcal{O}(\log n)$ à l'aide de l'exponentiation rapide.
   Un étudiant propose l'algorithme suivant :

```
function FastPow(x, n)
    if n == 0 then
        return 0
    end if
    half ← FastPow(x, n//2)
    if n is even then
        return half × half
    else
        return half × half × x
    end if
end function
```

   Quelle **modification minimale** rend l'algorithme correct tout en conservant la complexité $\mathcal{O}(\log n)$?
   ◯ Inverser les deux cas du `if n is even`.
   ◯ Remplacer `half * half * x` par `half * x * x`.
   ◯ Dans `FastPow`, retourner `1` lorsque `n == 0`.
   ◯ Ajouter un cas `if n == 1: return x`.

1.2 Structures de données et leurs utilisations

1. Un programme doit gérer une collection d'éléments sur laquelle les opérations suivantes sont très fréquentes :
   1. Ajouter un élément à la fin de la collection
   2. Accéder rapidement à l'élément du milieu
   3. Supprimer des éléments rapidement au début ou à la fin

   Quelle structure de données est la plus adaptée ?
   ◯ Tableau (*Array*)
   ◯ File (*Queue*)
   ◯ Liste doublement chaînée
   ◯ List simplement chaînée

2. Une application de messagerie doit gérer les conversations de la façon suivante :
   — Chaque message est ajouté en **temps réel** à la fin de la conversation
   — Les utilisateurs peuvent **supprimer les messages les plus récents**
   — L'application doit pouvoir **afficher rapidement le dernier message envoyé**
   Quelle structure de données est la plus appropriée pour ce comportement ?
   ◯ File (*Queue*)
   ◯ Pile (*Stack*)
   ◯ Tableau dynamique (*Array*)
   ◯ Liste simplement chaînée (*Singly Linked List*)

3. On veut parcourir un graphe en largeur à partir d'un nœud donné. Les sommets découverts doivent être traités dans l'ordre où ils ont été visités.

   Quelle structure de données faut-il utiliser pour gérer les sommets à explorer ?

   - ◯ Pile (*Stack*)
   - ◯ File (*Queue*)
   - ◯ Tas (*Heap*)
   - ◯ Tableau (*Array*)

1.3 Tri et recherche

1. On vous donne un tableau de n entiers non triés. Vous devez trouver **la plus petite différence absolue** entre deux éléments du tableau (et uniquement cette valeur).

   Quelle approche permet de résoudre ce problème plus efficacement que $\mathcal{O}(n^2)$ tout en garantissant le bon résultat ?

   - ◯ **Trier** le tableau puis parcourir une seule fois en comparant uniquement les paires **adjacentes**.
   - ◯ Calculer $|A_i - A_j|$ pour toutes les paires (i, j) et garder le minimum.
   - ◯ Insérer les éléments dans une table de hachage et vérifier s'il existe des doublons (si oui, renvoyer 0, sinon -1).
   - ◯ Pour chaque élément, faire une recherche binaire de l'élément le plus proche dans le tableau d'origine non trié.

1.4 Graphes et arbres

1. On considère un arbre composé de n sommets et de $n - 1$ arêtes. Si l'on supprime une **arête (edge)** de cet arbre, en combien de **sous-arbres** distincts le graphe est-il divisé ?

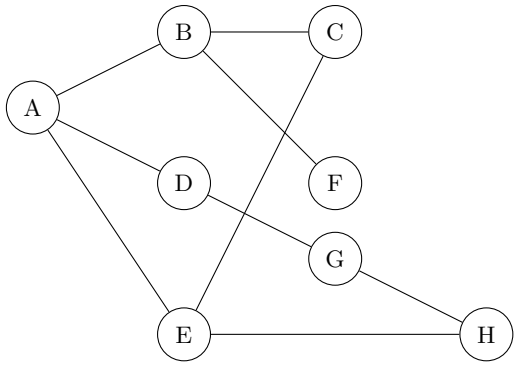
   - ◯ 0
   - ◯ 1
   - ◯ 2
   - ◯ $n - 2$

2. On considère un arbre non orienté à n sommets ($n > 1$). Le degré d'un sommet correspond au nombre d'arêtes (edges) connectées à ce sommet.

   On sait que chaque sommet a un degré d'au moins 1, sauf un seul sommet dont le degré est 0. Que peut-on conclure ?

   - ◯ Ce graphe n'est pas un arbre, car il contient un sommet isolé et donc plusieurs composantes connexes.
   - ◯ C'est un arbre où tous les sommets sont des feuilles.
   - ◯ C'est un arbre avec une seule composante connexe.
   - ◯ C'est un graphe orienté spécial où le degré 0 désigne la racine.

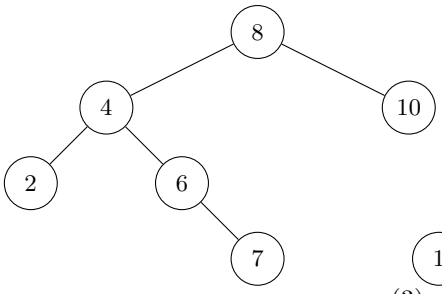
3. On considère le graphe non orienté ci-dessous. On effectue deux parcours à partir du sommet A :
   — Parcours en largeur (BFS)
   — Parcours en profondeur (DFS, pré-ordre)
   On suppose que, pour chaque sommet, ses voisins sont explorés *par ordre alphabétique*. Donnez l'ordre de visite (liste des sommets) pour chacun des parcours.



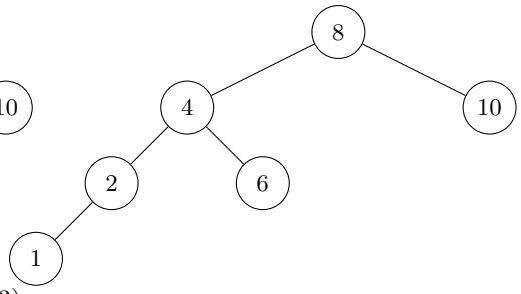
◯ **BFS :** A, B, E, D, C, F, H, G **DFS :** A, B, F, C, E, D, G, H
◯ **BFS :** A, D, B, E, G, C, F, H **DFS :** A, D, G, H, E, C, B, F
◯ **BFS :** A, B, D, E, F, C, H, G **DFS :** A, E, H, G, D, B, C, F
◯ **BFS :** A, B, D, E, C, F, G, H **DFS :** A, B, C, E, H, G, D, F

4. On considère les trois arbres binaires de recherche (BST) ci-dessous, représentés avec des *directed edges* du parent vers l'enfant. Lequel (ou lesquels) de ces arbres est un AVL ?

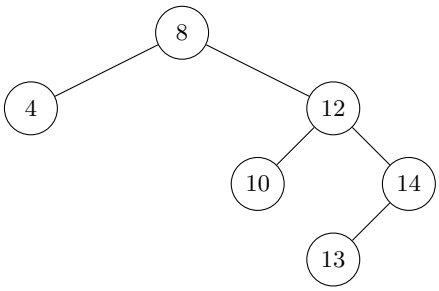
(1)



(2)



(3)

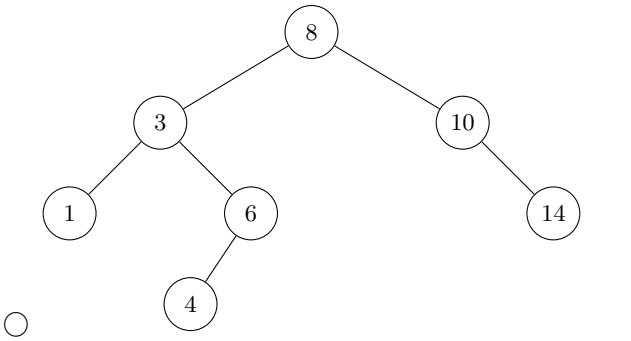


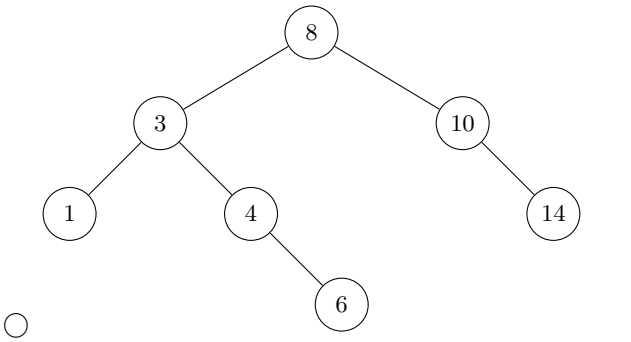
◯ Aucun
◯ (1) seulement
◯ (1) et (3)
◯ (2) et (3)

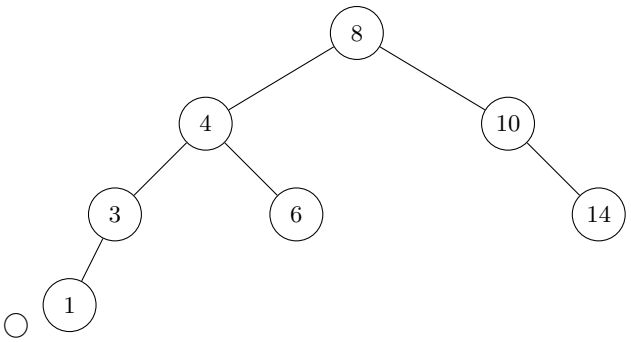
5. On lit successivement 7 entiers dans un tableau, de gauche à droite :

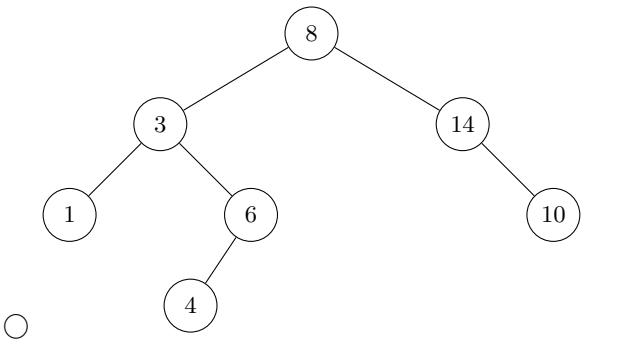
$$[\,8,\ 3,\ 10,\ 1,\ 6,\ 14,\ 4\,]$$

On construit un *Binary Search Tree* (BST) initialement vide en insérant les valeurs **dans cet ordre**, sans rééquilibrage (aucun contrôle AVL). Quel est le BST obtenu ?





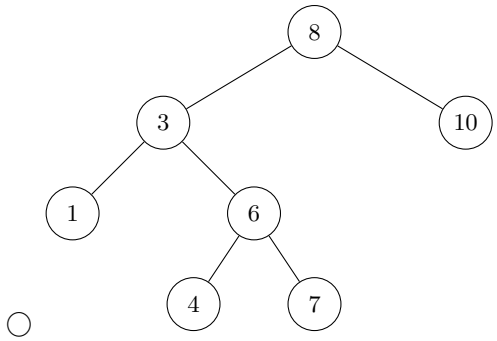


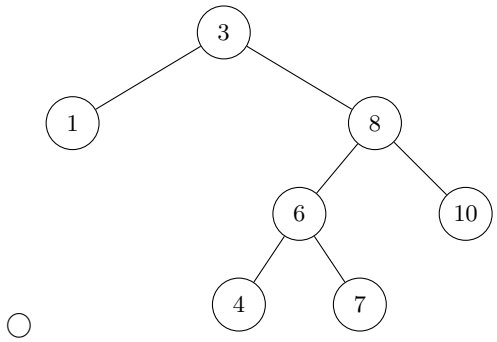


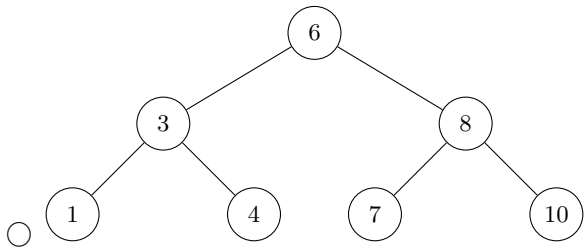
6. On lit successivement 7 entiers dans un tableau, de gauche à droite :

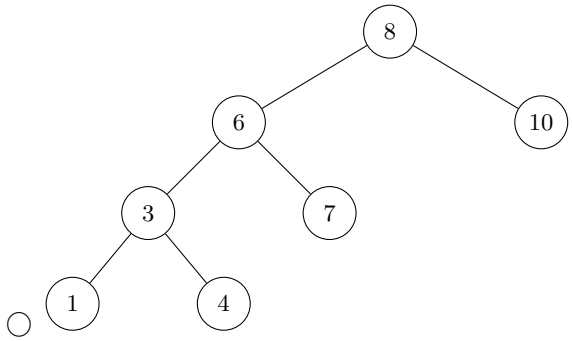
$$[\,8,\ 3,\ 10,\ 1,\ 6,\ 4,\ 7\,]$$

On construit un *AVL tree* (BST auto-équilibré) initialement vide en insérant les valeurs **dans cet ordre**, en effectuant un **rééquilibrage AVL après chaque insertion** (rotations si nécessaire). Quel est l'arbre final obtenu ?

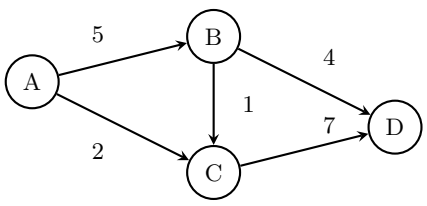
○ 

○ 

○ 

○ 

7. On considère le graphe orienté et pondéré ci-dessous. Déterminez la **liste d'adjacence correcte** correspondant à ce graphe.



○ A : [(B,5), (C,2)]
B : [(D,4)]
C : [(D,7)]
D : []

◯ A : [(B,5), (C,2)]
B : [(C,1), (D,4)]
C : [(D,7)]
D : []

◯ A : [(B,2), (C,5)]
B : [(D,7)]
C : [(D,4)]
D : []

◯ A : [(B,5)]
B : [(C,2), (D,4)]
C : [(D,7)]
D : []

8. Parmi les situations suivantes, laquelle est la mieux modélisée par un **graphe orienté et pondéré** ?
   - ◯ Un réseau social où chaque arête indique que deux utilisateurs sont amis.
   - ◯ Un système de recommandation reliant des films et des acteurs.
   - ◯ Un réseau routier où chaque arête indique la distance (ou le temps) entre deux villes, et le sens de circulation peut être différent dans chaque direction.
   - ◯ Un réseau électrique reliant des générateurs et des consommateurs sans notion de direction ni de coût.
9. Quelle structure de graphe est la plus appropriée pour modéliser les **relations entre les personnages d'un film**, où une arête relie deux personnages s'ils apparaissent ensemble dans une même scène ?
   - ◯ Arbre binaire
   - ◯ Graphe non orienté
   - ◯ Arbre général
   - ◯ Graphe orienté pondéré

BONNE CHANCE ! :)